

**LAPORAN PRAKTIKUM
ALGORITMA & STRUKTUR DATA
MODUL 6**



Graph and Tree

Oleh:

M. Rifky Almadani NIM. 2510817310018

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
JUNI
2026**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM ALGORITMA & STRUKTUR DATA
MODUL 6

Laporan Praktikum Algoritma & Struktur Data Modul 6: Graph and Tree ini disusun sebagai syarat lulus mata kuliah Praktikum Algoritma & Struktur Data. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : M. Rifky Almadani
NIM : 2510817310018

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Muhammad Fauzan Ahsani
NIM. 2310817310009

Ir. Muhamamd Alkaff, S.Kom, M.Kom, Ph.D
NIP. 198606132015041001

DAFTAR ISI

LEMBAR PENGESAHAN.....	ii
DAFTAR ISI	iii
DAFTAR GAMBAR.....	iv
DAFTAR TABEL	v
SOAL 1: Konversi Adjacency List ke Adjacency Matrix.....	1
A. Source Code.....	3
B. Output Program	4
C. Pembahasan	4
SOAL 2: Konversi Adjacency Matrix ke Adjacency List.....	7
A. Source Code.....	9
B. Output Program	10
C. Pembahasan	10
SOAL 3: BFS: Jarak Terpendek Keluar Labirin	12
A. Source Code.....	14
B. Output Program	15
C. Pembahasan	16
SOAL 4: DFS: Menghitung Semua Jalur Keluar Labirin	18
A. Source Code.....	20
B. Output Program	21
C. Pembahasan	21
SOAL 5: Traversal	24
A. Source Code.....	26
B. Output Program	28
C. Pembahasan	28
TAUTAN GIT	30

DAFTAR GAMBAR

Gambar 1 Screenshot Hasil Pengujian file soal1.cpp.....	4
Gambar 2 Screenshot Hasil Pengujian file soal2.cpp.....	10
Gambar 3 Screenshot Hasil Pengujian file soal3.cpp.....	15
Gambar 4 Screenshot Hasil Pengujian file soal4.cpp.....	21
Gambar 5 Screenshot Hasil Pengujian file soal5.cpp.....	28

DAFTAR TABEL

Tabel 1 Source Code file soal1.cpp.....	3
Tabel 2 Source Code file soal2.cpp.....	9
Tabel 3 Source Code file soal3.cpp.....	14
Tabel 4 Source Code file soal4.cpp.....	20
Tabel 5 Source Code file soal5.cpp.....	26

SOAL 1: Konversi Adjacency List ke Adjacency Matrix

Time Limit: 1.0 s

Memory Limit: 64 MB

Deskripsi Soal

Diberikan sebuah **directed weighted graph** yang terdiri dari N vertex dan M edge. Setiap vertex memiliki label berupa satu karakter unik. Graph diberikan dalam bentuk daftar edge.

Setiap edge ditulis sebagai U,V,W , yang berarti terdapat edge berarah dari vertex U menuju vertex V dengan bobot W . Karena graph bersifat berarah, edge U,V,W tidak otomatis berarti terdapat edge dari V menuju U .

Tugas Anda adalah mengubah representasi graph tersebut menjadi adjacency matrix. Jika terdapat edge dari vertex ke- i menuju vertex ke- j , maka nilai matrix pada baris i dan kolom j adalah bobot edge tersebut. Jika tidak terdapat edge, nilainya 0.

Format Input

Input diberikan dalam format berikut.

```
N
V1 V2 ... VN
M
U1 V1 W1
U2 V2 W2
...
UM VM WM
```

Keterangan:

- N adalah jumlah vertex.
- V_i adalah label vertex ke- i .
- M adalah jumlah edge.
- U_i dan V_i adalah vertex asal dan vertex tujuan pada edge ke- i .
- W_i adalah bobot edge ke- i .

Format Output

Cetak adjacency matrix dengan format berikut.

Adjacency Matrix:

```
V1 V2 ... VN
V1 a111 a12 ... a1N
V2 a21 a22 ... a2N
...
VN aN1 aN2 ... aNN
```

Nilai a_{ij} adalah bobot edge dari vertex V_i menuju vertex V_j . Jika edge tersebut tidak ada, nilai a_{ij} adalah 0.

Batasan

- $2 \leq N \leq 10$
- $0 \leq M \leq N * (N - 1)$
- $1 \leq W_i \leq 1000$
- Label vertex berupa karakter alfabet kapital A sampai Z.
- Semua label vertex dijamin unik.
- Graph tidak memiliki self-loop.
- Graph tidak memiliki edge dupikat dengan arah yang sama.
- Graph bersifat berarah dan berbobot.

Contoh Kasus

Input	Output
5 A B C D E 6 A B 4 A C 2 B D 7 C B 1 C E 6 D C 3	Adjacency Matrix: A B C D E A 0 4 2 0 0 B 0 0 0 7 0 C 0 1 0 0 6 D 0 0 3 0 0 E 0 0 0 0 0

Penjelasan Contoh

Edge A B 4 menghasilkan nilai 4 pada baris A kolom B. Karena graph berarah, nilai pada baris B kolom A tetap 0 selama tidak ada edge dari B menuju A.

Edge D C 3 menghasilkan nilai 3 pada baris D kolom C. Vertex E tidak memiliki edge keluar, sehingga seluruh nilai pada baris E adalah 0.

A. Source Code

Tabel 1 Source Code file soal1.cpp

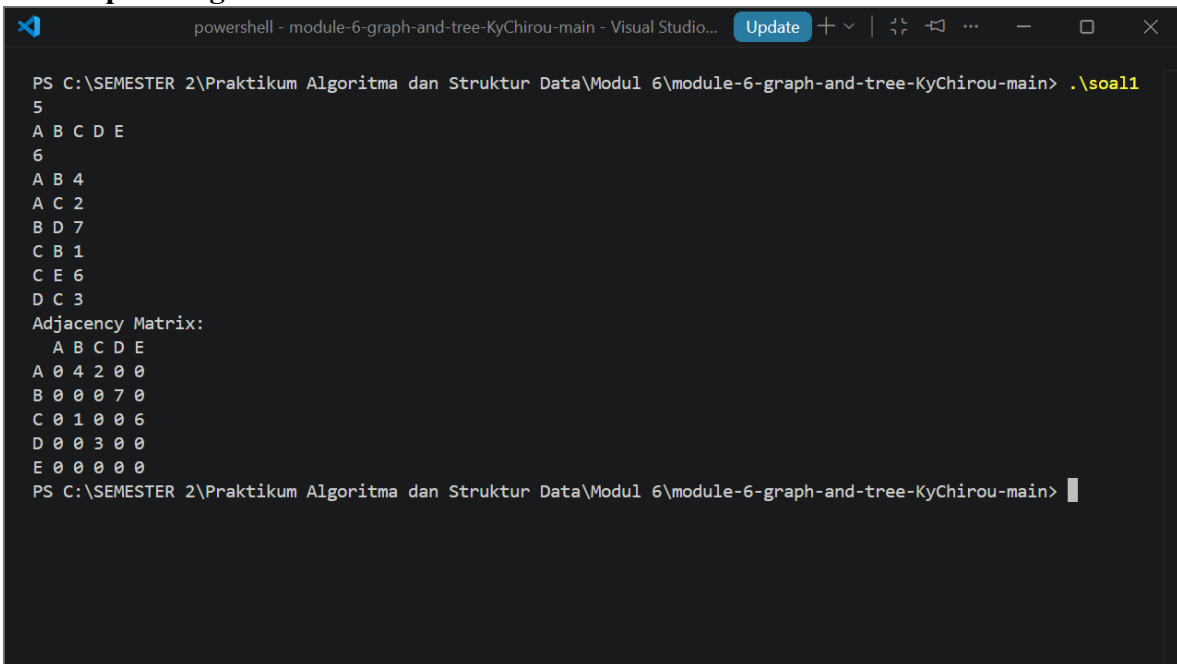
```
1  #include <iostream>
2  #include <vector>
3  #include <map>
4  using namespace std;
5
6  int main() {
7      int N;
8      if (!(cin >> N)) return 0;
9
10     vector<char> vertices(N);
11     map<char, int> v_index;
12
13     for (int i = 0; i < N; ++i) {
14         cin >> vertices[i];
15         v_index[vertices[i]] = i;
16     }
17
18     int M;
19     cin >> M;
20
21     vector<vector<int>> matrix(N, vector<int>(N, 0));
22     for (int i = 0; i < M; ++i) {
23         char u, v;
24         int w;
25         cin >> u >> v >> w;
26         matrix[v_index[u]][v_index[v]] = w;
27     }
28
29     cout << "Adjacency Matrix:\n ";
30     for (int i = 0; i < N; ++i) cout << vertices[i] << (i
31 == N - 1 ? "" : " ");
32     cout << "\n";
33
34     for (int i = 0; i < N; ++i) {
35         cout << vertices[i] << " ";
```

```

35         for (int j = 0; j < N; ++j) {
36             cout << matrix[i][j] << (j == N - 1 ? "" : "
37         );
38         }
39         cout << "\n";
40     }
41     return 0;
42 }

```

B. Output Program



```

PS C:\SEMESTER 2\Praktikum Algoritma dan Struktur Data\Modul 6\module-6-graph-and-tree-KyChirou-main> .\soal1
5
A B C D E
6
A B 4
A C 2
B D 7
C B 1
C E 6
D C 3
Adjacency Matrix:
  A B C D E
A 0 4 2 0 0
B 0 0 0 7 0
C 0 1 0 0 6
D 0 0 3 0 0
E 0 0 0 0 0
PS C:\SEMESTER 2\Praktikum Algoritma dan Struktur Data\Modul 6\module-6-graph-and-tree-KyChirou-main>

```

Gambar 1 Screenshot Hasil Pengujian file soal1.cpp

C. Pembahasan

Kode ini merupakan implementasi Adjacency Matrix yang dirancang untuk merepresentasikan sebuah graf berbobot dan berarah. Secara garis besar, program bekerja dengan menerima input berupa jumlah vertex, label setiap vertex, jumlah edge, serta bobot yang menghubungkan masing-masing edge. Output yang dihasilkan adalah visualisasi adjacency matrix yang secara akurat memetakan hubungan antar vertex beserta bobotnya. Untuk mencapai fungsionalitas tersebut, program memanfaatkan dua struktur data utama dari Standard Template Library (STL) C++, yaitu vector sebagai wadah penyimpanan nama vertex dan matriks, serta map sebagai mekanisme

pengindeksan yang memetakan label karakter setiap vertex ke koordinat numerik dalam matriks.

Algoritma ini dimulai dengan deklarasi `vector<char> vertices` untuk menampung seluruh label vertex dan `map<char, int> v_index` untuk menyimpan posisi atau indeks setiap vertex. Dengan menggunakan `map`, program dapat secara langsung mengakses posisi suatu vertex dalam matriks tanpa harus melakukan pencarian berulang kali; misalnya, jika vertex yang dimasukkan adalah A, B, C, D, dan E, maka `map` secara otomatis memetakan A→0, B→1, C→2, D→3, dan E→4. Tahap pengisian vertex ini membutuhkan kompleksitas waktu sebesar $O(N)$ dan kompleksitas ruang sebesar $O(N)$ karena menyimpan N buah nama vertex beserta pemetaannya. Selanjutnya, program menyiapkan struktur `vector<vector<int>> matrix` berukuran $N \times N$ yang diinisialisasi dengan nilai 0, sebagai penanda bahwa pada tahap awal tidak terdapat hubungan antar vertex. Inisialisasi matriks ini memiliki kompleksitas waktu $O(N^2)$ dan kompleksitas ruang $O(N^2)$ untuk penyediaan memori grid.

Setelah struktur matriks terbentuk, program memasuki tahap input edge melalui perulangan sebanyak M kali. Pada setiap iterasi, program membaca titik asal, titik tujuan, dan bobot, kemudian menggunakan `v_index` untuk mencari koordinat matriks yang sesuai dan menyimpan bobot tersebut. Contohnya, jika diinputkan edge A B 4, maka nilai 4 akan ditempatkan pada posisi `matrix[v_index['A']][v_index['B']]`. Karena proses pengisian ini merupakan operasi akses langsung (direct addressing), setiap operasi memiliki kompleksitas waktu $O(1)$, sehingga total kompleksitas untuk M buah edge adalah $O(M)$. Tahap terakhir adalah proses pencetakan matriks menggunakan dua perulangan bersarang (nested loop). Program mencetak label vertex sebagai header, diikuti dengan isi setiap baris matriks. Proses pencetakan ini memiliki kompleksitas waktu $O(N^2)$ karena harus mengunjungi setiap sel pada matriks.

Secara keseluruhan, jika ditinjau dari sisi performa, kompleksitas waktu dari program ini adalah $O(N + M + N^2)$. Komponen $O(N)$ muncul dari inisialisasi vertex, $O(M)$ berasal dari input edge, dan $O(N^2)$ didominasi oleh proses inisialisasi serta pencetakan matriks. Mengingat $O(N^2)$ merupakan komponen dengan pertumbuhan terbesar, maka

kompleksitas waktu dominan program ini dapat disederhanakan menjadi $O(N^2)$. Sementara itu, untuk kompleksitas ruang, kebutuhan memori terbesar dialokasikan untuk adjacency matrix berukuran $N \times N$, sehingga program ini memiliki kompleksitas ruang sebesar $O(N^2)$. Pendekatan ini membuktikan efektivitas Space-Time Tradeoff dalam graf; meskipun penggunaan matriks menuntut alokasi ruang yang cukup besar, program mampu memberikan kecepatan akses rute pada waktu konstan $O(1)$, yang sangat efisien untuk memvalidasi keberadaan maupun bobot hubungan antar vertex dalam graf.

SOAL 2: Konversi Adjacency Matrix ke Adjacency List

Time Limit: 1.0 s

Memory Limit: 64 MB

Deskripsi Soal

Diberikan sebuah **directed weighted graph** yang direpresentasikan dalam bentuk **adjacency matrix** berukuran $N \times N$. Setiap vertex memiliki label berupa satu karakter unik.

Nilai matrix pada baris i dan kolom j menunjukkan bobot edge dari vertex ke- i menuju vertex ke- j . Jika nilainya 0, berarti tidak terdapat edge dari vertex ke- i menuju vertex ke- j .

Tugas Anda adalah mengubah adjacency matrix tersebut menjadi **adjacency list**. Untuk setiap vertex, cetak semua vertex tujuan yang dapat dicapai langsung beserta bobot edge-nya. Urutan vertex tujuan mengikuti urutan label pada input.

Format Input

Input diberikan dalam format berikut.

```
N
V1 V2 ... VN
A11 A12 ... A1N
A21 A22 ... A2N
...
AN1 AN2 ... ANN
```

Keterangan:

- N adalah jumlah vertex.
- V_i adalah label vertex ke- i .
- A_{ij} adalah nilai matrix pada baris ke- i dan kolom ke- j .
- Jika $A_{ij} > 0$, maka terdapat edge dari V_i menuju V_j dengan bobot A_{ij} .
- Jika $A_{ij} = 0$, maka tidak terdapat edge dari V_i menuju V_j .

Format Output

Cetak adjacency list untuk setiap vertex dengan format berikut.

Adjacency List:

V1 -> (X1, w1), (X2, w2), ...

V2 -> (X1, w1), (X2, w2), ...

...

VN → ...

Jika sebuah vertex tidak memiliki edge keluar, cetak tanda – setelah simbol -> .

Constraints

- $2 \leq N \leq 10$
- $0 \leq A_{ij} \leq 1000$
- Label vertex berupa karakter alfabet kapital A sampai Z.
- Semua label vertex dijamin unik.
- Nilai diagonal utama matrix dijamin 0.
- Nilai 0 berarti tidak ada edge.
- Nilai positif berarti bobot edge.
- Matrix merepresentasikan graph berarah dan berbobot.

Contoh Kasus

Input	Output
5 A B C D E 0 4 2 0 0 0 0 0 7 0 0 1 0 0 6 0 0 3 0 0 0 0 0 0 0	Adjacency List: A -> (B, 4), (C, 2) B -> (D, 7) C -> (B, 1), (E, 6) D -> (C, 3) E ->

Penjelasan Contoh

Pada baris vertex A, nilai positif muncul pada kolom B dan C. Karena itu, adjacency list untuk A adalah (B,4), (C,2).

Pada baris vertex E, semua nilai bernilai 0, sehingga vertex E tidak memiliki edge keluar dan output-nya adalah E -> -.

A. Source Code

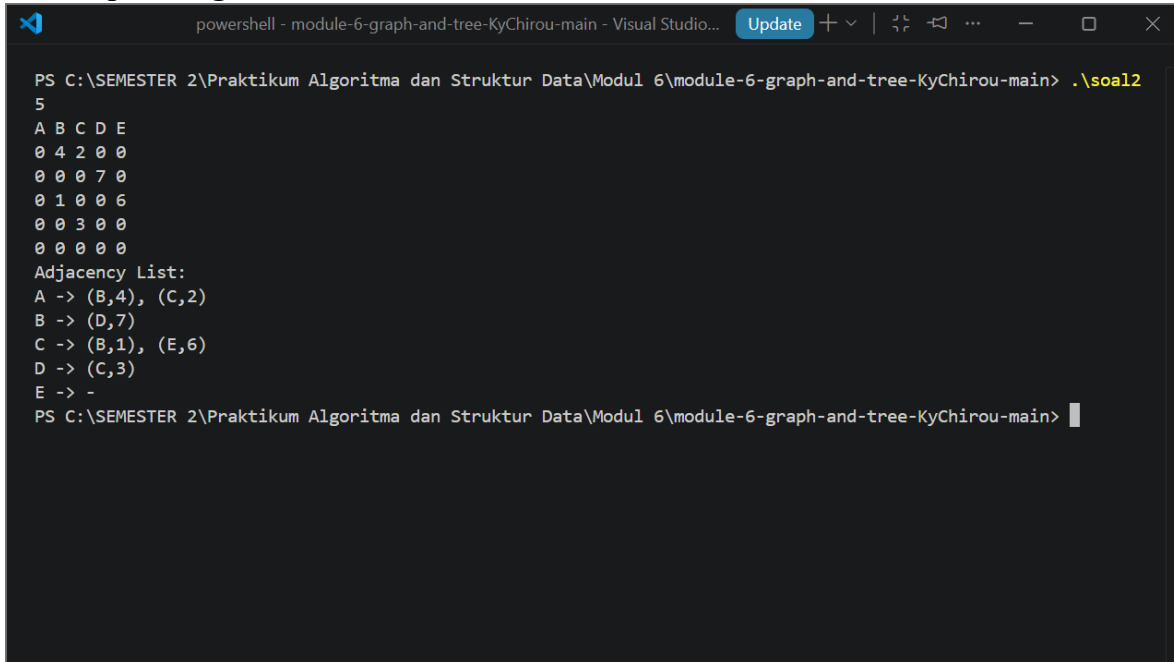
Tabel 2 Source Code file soal2.cpp

```

1  #include <iostream>
2  #include <vector>
3  using namespace std;
4
5  int main() {
6      int N;
7      if (!(cin >> N)) return 0;
8
9      vector<char> vertices(N);
10     for (int i = 0; i < N; ++i) {
11         cin >> vertices[i];
12     }
13
14     vector<vector<int>> matrix(N, vector<int>(N));
15     for (int i = 0; i < N; ++i) {
16         for (int j = 0; j < N; ++j) {
17             cin >> matrix[i][j];
18         }
19     }
20
21     cout << "Adjacency List:\n";
22     for (int i = 0; i < N; ++i) {
23         cout << vertices[i] << " -> ";
24         bool hasEdge = false;
25
26         for (int j = 0; j < N; ++j) {
27             if (matrix[i][j] > 0) {
28                 if (hasEdge) cout << ", ";
29                 cout << "(" << vertices[j] << ", " <<
matrix[i][j] << ")";
30                 hasEdge = true;
31             }
32         }
33
34         if (!hasEdge) cout << "-";
35         cout << "\n";
36     }
37
38     return 0;
39 }

```

B. Output Program



```
PS C:\SEMESTER 2\Praktikum Algoritma dan Struktur Data\Modul 6\module-6-graph-and-tree-KyChirou-main> .\soal2
5
A B C D E
0 4 2 0 0
0 0 0 7 0
0 1 0 0 6
0 0 3 0 0
0 0 0 0 0
Adjacency List:
A -> (B,4), (C,2)
B -> (D,7)
C -> (B,1), (E,6)
D -> (C,3)
E -> -
PS C:\SEMESTER 2\Praktikum Algoritma dan Struktur Data\Modul 6\module-6-graph-and-tree-KyChirou-main>
```

Gambar 2 Screenshot Hasil Pengujian file soal2.cpp

C. Pembahasan

Kode ini dirancang untuk melaksanakan operasi transformasi representasi graf dari format adjacency matrix kembali menjadi bentuk adjacency list. Transformasi ini berfokus pada ekstraksi data dari struktur grid dua dimensi menjadi struktur daftar sekuensial yang merangkum setiap titik asal beserta himpunan titik tujuannya. Asumsi dasar yang mendasari rancangan algoritma ini meliputi ketersediaan nilai bobot positif sebagai penanda mutlak keberadaan rute (edge), sedangkan sel dengan nilai nol secara eksplisit direpresentasikan sebagai ketiadaan jalur antar simpul. Selain itu, graf diposisikan sebagai graf berarah (directed graph) berbobot dengan label vertex unik yang disimpan menggunakan karakter abjad tunggal, di mana urutan label penjelajahan dijamin sejajar dengan indeks matriks dari input yang diberikan.

Dalam implementasinya, program ini secara ekstensif menggunakan struktur data kontainer dinamis dari pustaka Standard Template Library (STL) bahasa C++. Eksekusi diawali dengan instruksi validasi `if (!(cin >> N))` untuk mengamankan alur eksekusi program dari galat pembacaan akibat data masukan yang kosong. Kumpulan label titik direkam secara linear menggunakan tipe data `vector<char>`, sementara

data topologi graf ditampung ke dalam memori dua dimensi melalui deklarasi `vector<vector<int>> matrix` berukuran $N \times N$. Algoritma penjelajahan (traversal) utama dibangun menggunakan perulangan ganda (nested loop), perulangan tingkat pertama mewakili indeks titik asal, sedangkan perulangan tingkat kedua bertugas memindai pergerakan indeks titik tujuan. Di dalam blok perulangan tersebut, program membedah isi memori dengan mengevaluasi kondisi `if (matrix[i][j] > 0)` untuk menyaring dan mencetak titik tujuan beserta nilai bobotnya. Program juga mengaplikasikan teknik flagging berbasis state menggunakan variabel logika `hasEdge` untuk mengontrol penempatan tanda koma secara dinamis, serta mencetak karakter tanda hubung apabila kondisi mendeteksi bahwa sebuah vertex tidak memiliki rute keluar sama sekali.

Algoritma transformasi ini menghasilkan kompleksitas waktu yang tertahan secara absolut pada laju $O(N^2)$. Kinerja komputasi ini terbentuk murni dari kewajiban program untuk memindai setiap sel pada matriks bujursangkar berukuran $N \times N$ tanpa terkecuali, terlepas dari fakta apakah matriks tersebut padat dengan rute atau didominasi oleh nilai nol. Dari perspektif konsumsi memori internal, program ini memakan kompleksitas ruang sebesar $O(N^2)$ dikarenakan sistem wajib mengalokasikan porsi memori secara utuh untuk memetakan seluruh koordinat matriks sebelum proses ekstraksi dimulai. Secara konseptual, implementasi ini memberikan pembuktian fungsional terhadap evaluasi metrik efisiensi penyimpanan (storage efficiency metrics). Melalui konversi dan pemetaan senarai ketetanggaan ini, struktur data yang diekstrak mampu menampilkan reduksi elemen kosong secara signifikan. Penelusuran sekuensial pada algoritma membuktikan kapabilitas program dalam meringkas identitas rute V_i menuju V_j dengan bobot positif A_{ij} menjadi format visual berantai yang murni hanya berisi jalur-jalur relevan dan valid.

SOAL 3: BFS: Jarak Terpendek Keluar Labirin

Time Limit: 1.0 s

Memory Limit: 64 MB

Deskripsi Soal

Sebuah labirin direpresentasikan sebagai grid berukuran $R \times C$. Setiap sel pada grid memiliki nilai sebagai berikut.

- 0 menyatakan jalan dapat dilewati.
- 1 menyatakan dinding yang tidak dapat dilewati.

Diberikan posisi awal dan posisi tujuan. Anda harus menentukan jumlah langkah minimum dari posisi awal menuju posisi tujuan menggunakan algoritma **Breadth-First Search (BFS)**.

Dari sebuah sel, Anda hanya boleh bergerak ke empat arah: atas, bawah, kiri, dan kanan. Pergerakan diagonal tidak diperbolehkan. Beberapa jalur pada labirin dapat berupa jalan buntu, sehingga algoritma harus tetap memproses kemungkinan jalur secara sistematis.

Format Input

Input diberikan dalam format berikut.

```
R C
G11 G12 ... G1C
G21 G22 ... G2C
...
GR1 GR2 ... GRC
SR SC
FR FC
```

Keterangan:

- R adalah jumlah baris grid.
- C adalah jumlah kolom grid.
- G_{ij} adalah nilai sel pada baris ke- i dan kolom ke- j .
- (S_R, S_C) adalah posisi awal.
- (F_R, F_C) adalah posisi tujuan.

Indeks baris dan kolom dimulai dari 0.

Format Output

Cetak satu bilangan bulat.

X

Keterangan

- X adalah jumlah langkah minimum dari posisis awal ke posisi tujuan.
- Jika posisi tujuan tidak dapat dicapai, cetak -1.

Constraints

- $1 \leq R \leq 100$
- $1 \leq C \leq 100$
- G_{ij} hanya bernilai 0 atau 1.
- $0 \leq S_R < R$
- $0 \leq S_C < C$
- $0 \leq F_R < R$
- $0 \leq F_C < C$
- Sel start dan finish dijamin bernilai 0.
- Pergerakan hanya boleh dilakukan ke sel bernilai 0.
- Algoritma yang digunakan wajib BFS.

Contoh Kasus

Input	Output
8 10 0 0 0 0 1 0 0 0 0 0 1 1 1 0 1 0 1 1 1 0 0 0 1 0 0 0 0 0 1 0 0 1 1 1 1 1 1 0 1 0 0 0 0 0 0 0 1 0 0 0 1 1 1 1 1 0 1 1 1 0 0 0 0 0 1 0 0 0 1 0 0 1 1 0 0 0 1 0 0 0 0 0 7 9	16

Penjelasan Contoh

Posisi awal berada pada (0,0) dan posisi tujuan berada pada (7,9). Labirin memiliki beberapa percabangan dan jalan buntu, misalnya cabang menuju area kiri bawah dan cabang menuju sel (2,1).

Dengan BFS, setiap sel diproses berdasarkan jarak terdekat dari start. Jarak minimum dari (0,0) ke (7,9) adalah 16 langkah, sehingga output yang dicetak hanya 16.

A. Source Code

Tabel 3 Source Code file soal3.cpp

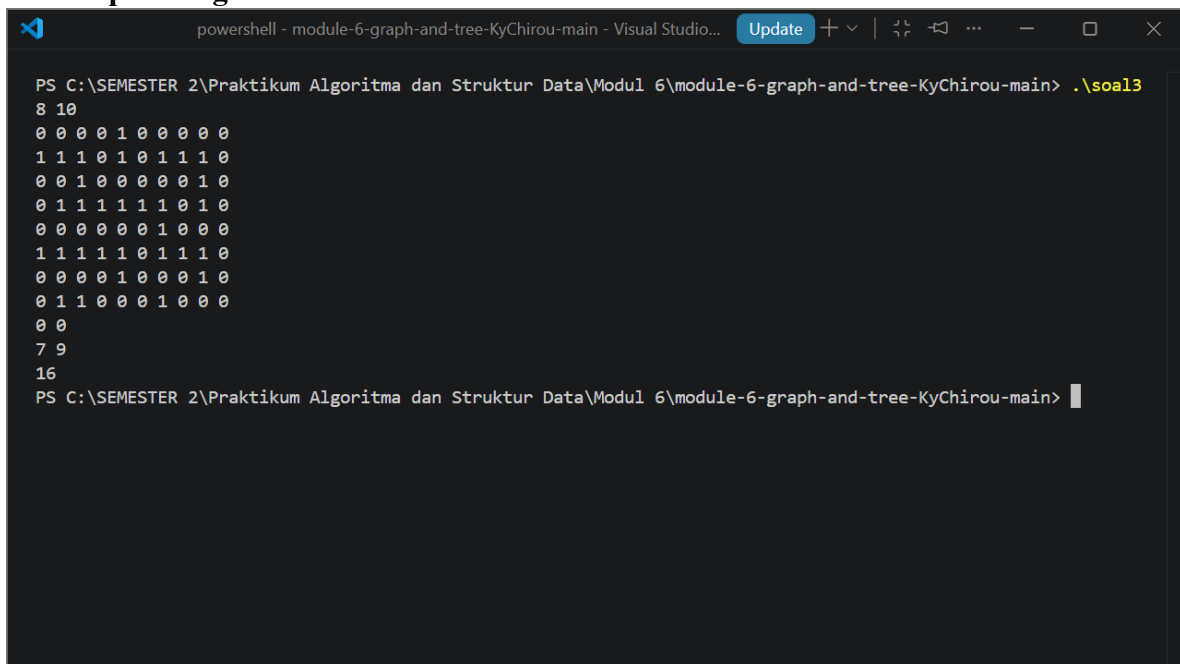
```
1  #include <iostream>
2  #include <vector>
3  #include <queue>
4  using namespace std;
5
6  struct State {
7      int r, c, dist;
8  };
9
10 int main() {
11     int R, C;
12     if (!(cin >> R >> C)) return 0;
13
14     vector<vector<int>> grid(R, vector<int>(C));
15     for (int i = 0; i < R; ++i) {
16         for (int j = 0; j < C; ++j) {
17             cin >> grid[i][j];
18         }
19     }
20
21     int SR, SC, FR, FC;
22     cin >> SR >> SC >> FR >> FC;
23
24     vector<vector<bool>> visited(R, vector<bool>(C,
false));
25     queue<State> q;
26
27     q.push({SR, SC, 0});
28     visited[SR][SC] = true;
29
30     int dr[] = {-1, 1, 0, 0};
31     int dc[] = {0, 0, -1, 1};
32
33     int ans = -1;
```

```

34
35     while (!q.empty()) {
36         State curr = q.front();
37         q.pop();
38
39         if (curr.r == FR && curr.c == FC) {
40             ans = curr.dist;
41             break;
42         }
43
44         for (int i = 0; i < 4; ++i) {
45             int nr = curr.r + dr[i];
46             int nc = curr.c + dc[i];
47
48             if (nr >= 0 && nr < R && nc >= 0 && nc < C &&
grid[nr][nc] == 0 && !visited[nr][nc]) {
49                 visited[nr][nc] = true;
50                 q.push({nr, nc, curr.dist + 1});
51             }
52         }
53     }
54
55     cout << ans << "\n";
56     return 0;
57 }

```

B. Output Program



```

powershell - module-6-graph-and-tree-KyChirou-main - Visual Studio... Update
PS C:\SEMESTER 2\Praktikum Algoritma dan Struktur Data\Modul 6\module-6-graph-and-tree-KyChirou-main> .\soal3
8 10
0 0 0 0 1 0 0 0 0 0
1 1 1 0 1 0 1 1 1 0
0 0 1 0 0 0 0 0 1 0
0 1 1 1 1 1 1 0 1 0
0 0 0 0 0 0 1 0 0 0
1 1 1 1 1 0 1 1 1 0
0 0 0 0 1 0 0 0 1 0
0 1 1 0 0 0 1 0 0 0
0 0
7 9
16
PS C:\SEMESTER 2\Praktikum Algoritma dan Struktur Data\Modul 6\module-6-graph-and-tree-KyChirou-main>

```

Gambar 3 Screenshot Hasil Pengujian file soal3.cpp

C. Pembahasan

Kode ini dirancang untuk menyelesaikan permasalahan pencarian jarak lintasan terpendek dari titik awal menuju titik tujuan dalam labirin menggunakan algoritma Breadth-First Search (BFS). Labirin direpresentasikan sebagai grid dua dimensi berukuran $R \times C$, di mana sel bernilai nol mengindikasikan ruang terbuka yang dapat dilalui dan sel bernilai satu merepresentasikan penghalang yang tidak dapat diakses. Algoritma BFS dipilih karena sifatnya yang mengeksplorasi graf atau grid secara melebar, yakni mengunjungi setiap simpul berdasarkan urutan jarak terdekat dari titik awal sebelum melangkah ke jarak yang lebih jauh. Asumsi dasar yang mengikat perancangan algoritma ini meliputi pergerakan yang bersifat ortogonal ke empat arah mata angin (atas, bawah, kiri, dan kanan), serta kondisi bahwa setiap langkah pergerakan memiliki bobot biaya seragam bernilai satu. Program juga dibangun di bawah asumsi logis bahwa sel titik keberangkatan dan tujuan akhir selalu berada pada koordinat terbuka bernilai nol.

Dalam implementasinya, program merealisasikan penjelajahan melebar ini dengan mendayagunakan memori antrean dinamis (queue) dari Standard Template Library (STL) C++. Untuk mempermudah manajemen data, program membungkus informasi koordinat baris, kolom, dan akumulasi jarak ke dalam struktur data kustom bernama `struct State`. Proses inisialisasi diawali dengan pembacaan dimensi grid dan pengisian seluruh elemen sel menggunakan perulangan ganda (nested loop). Program kemudian menyiapkan matriks penanda `visited` bertipe boolean berukuran $R \times C$ yang diinisialisasi dengan nilai `false` sebagai indikator bahwa sel tersebut belum pernah dijamah oleh algoritma. Pada fase eksekusi BFS, koordinat awal dimasukkan ke dalam antrean, dan selama antrean tidak dalam keadaan kosong, sistem akan mengekstrak elemen terdepan untuk diinspeksi. Sistem kemudian memantik ekspansi pencarian ke empat arah sekaligus menggunakan manipulasi array bantuan `dr` dan `dc`. Validasi ketat diterapkan untuk setiap sel tetangga: sel baru hanya akan diproses jika koordinatnya berada dalam batas dimensi grid, merupakan jalan terbuka, dan belum pernah dikunjungi sebelumnya.

Penelusuran BFS pada program ini mengonsumsi performa dengan kompleksitas waktu sebesar $O(R \times C)$. Estimasi ini mewakili skenario terburuk (worst-case scenario), di mana algoritma dipaksa mengunjungi setiap sel pada grid tepat satu kali sebelum mencapai target tujuan. Dari perspektif utilisasi memori, kompleksitas ruang yang dibutuhkan adalah $O(R \times C)$, yang dialokasikan untuk menampung grid utama, matriks *visited*, serta struktur antrean yang dalam kondisi puncak dapat menampung sekumpulan posisi sel secara simultan. Pendekatan ini memberikan pembuktian metrik yang valid terhadap tuntutan persoalan, yakni penentuan jarak tempuh minimum. Melalui sifat penelusuran level-order yang melebar merata ke segala arah, titik tujuan yang berhasil dijamah pertama kali oleh algoritma BFS dijamin secara matematis sebagai representasi rekam langkah yang paling minimal, sekaligus memvalidasi bahwa efisiensi algoritma ini mampu menanggulangi kompleksitas labirin dengan presisi yang optimal.

SOAL 4: DFS: Menghitung Semua Jalur Keluar Labirin

Time Limit: 1.0 s

Memory Limit: 64 MB

Deskripsi Soal

Diberikan sebuah labirin berukuran $R \times C$ dengan aturan sel yang sama seperti soal sebelumnya. Anda harus menghitung banyaknya jalur sederhana dari posisi awal menuju posisi tujuan menggunakan algoritma **Depth-First Search (DFS)** rekursif.

Sebuah jalur disebut sederhana jika tidak mengunjungi sel yang sama lebih dari satu kali dalam jalur yang sama. Dari sebuah sel, Anda hanya boleh bergerak ke empat arah: atas, bawah, kiri, dan kanan

Labirin dapat memiliki beberapa percabangan dan jalan buntu. Karena itu, DFS harus melakukan proses **backtracking** agar dapat mencoba semua kemungkinan jalur yang valid.

Format Input

Input diberikan dalam format berikut.

```
R C
G11 G12 ... G1C
G21 G22 ... G2C
...
GR1 GR2 ... GRC
SR SC
FR FC
```

Keterangan:

- R adalah jumlah baris grid.
- C adalah jumlah kolom grid.
- G_{ij} adalah nilai sel pada baris ke- i dan kolom ke- j .
- (S_R, S_C) adalah posisi awal.
- (F_R, F_C) adalah posisi tujuan.

Indeks baris dan kolom dimulai dari 0.

Format Output

Cetak satu bilangan bulat.

T

Keterangan

- T adalah jumlah jalur sederhana dari posisi awal ke posisi tujuan.
- Jika tidak ada jalur yang dapat dilewati, cetak 0.

Constraints

- $1 \leq R \leq 7$
- $1 \leq C \leq 7$
- G_{ij} hanya bernilai 0 atau 1.
- $0 \leq S_R < R$
- $0 \leq S_C < C$
- $0 \leq F_R < R$
- $0 \leq F_C < C$
- Sel start dan finish dijamin bernilai 0.
- Pergerakan hanya boleh dilakukan ke sel bernilai 0.
- Satu jalur tidak boleh mengunjungi sel yang sama lebih dari satu kali.
- Jumlah jalur valid pada test case dijamin tidak lebih dari 100000.
- Algoritma yang digunakan wajib DFS rekursif dengan backtracking.

Contoh Kasus

Input	Output
5 6 0 0 0 1 0 0 0 1 0 0 0 1 0 1 0 1 0 0 0 0 0 1 1 0 1 1 0 0 0 0 0 0 4 5	4

Penjelasan Contoh

Posisi awal berada pada (0,0) dan posisi tujuan berada pada (4,5). Labirin memiliki lebih dari satu jalur valid serta beberapa cabang yang tidak menghasilkan solusi, misalnya cabang menuju sel (0,5).

Dengan DFS dan backtracking, seluruh jalur sederhana dari start ke finish dapat dihitung. Pada contoh ini terdapat 4 jalur valid, sehingga output yang dicetak hanya 4.

A. Source Code

Tabel 4 Source Code file soal4.cpp

```
1  #include <iostream>
2  #include <vector>
3  using namespace std;
4
5  int R, C;
6  int SR, SC, FR, FC;
7  vector<vector<int>> grid;
8  vector<vector<bool>> visited;
9  int totalPaths = 0;
10
11 int dr[] = {-1, 1, 0, 0};
12 int dc[] = {0, 0, -1, 1};
13
14 void dfs(int r, int c) {
15     if (r == FR && c == FC) {
16         totalPaths++;
17         return;
18     }
19
20     visited[r][c] = true;
21
22     for (int i = 0; i < 4; ++i) {
23         int nr = r + dr[i];
24         int nc = c + dc[i];
25
26         if (nr >= 0 && nr < R && nc >= 0 && nc < C &&
27             grid[nr][nc] == 0 && !visited[nr][nc]) {
28             dfs(nr, nc);
29         }
30
31         visited[r][c] = false;
32     }
33 }
```

```

34 int main() {
35     if (!(cin >> R >> C)) return 0;
36
37     grid.assign(R, vector<int>(C));
38     visited.assign(R, vector<bool>(C, false));
39
40     for (int i = 0; i < R; ++i) {
41         for (int j = 0; j < C; ++j) {
42             cin >> grid[i][j];
43         }
44     }
45
46     cin >> SR >> SC >> FR >> FC;
47
48     dfs(SR, SC);
49
50     cout << totalPaths << "\n";
51     return 0;
52 }

```

B. Output Program

```

powershell - module-6-graph-and-tree-KyChirou-main - Visual Studio... Update
PS C:\SEMESTER 2\Praktikum Algoritma dan Struktur Data\Modul 6\module-6-graph-and-tree-KyChirou-main> .\soal4
5 6
0 0 0 1 0 0
0 1 0 0 0 1
0 1 0 1 0 0
0 0 0 1 1 0
1 1 0 0 0 0
0 0
4 5
4
PS C:\SEMESTER 2\Praktikum Algoritma dan Struktur Data\Modul 6\module-6-graph-and-tree-KyChirou-main>

```

Gambar 4 Screenshot Hasil Pengujian file soal4.cpp

C. Pembahasan

Kode ini dibangun untuk menyelesaikan permasalahan perhitungan total jalur sederhana dari titik awal menuju titik tujuan di dalam labirin dua dimensi dengan mengimplementasikan algoritma Depth-First Search (DFS) yang dikombinasikan dengan

teknik backtracking. Labirin direpresentasikan sebagai grid berukuran $R \times C$, di mana sel bernilai nol berfungsi sebagai ruang yang dapat dilalui dan nilai satu sebagai rintangan. Berbeda dengan pendekatan BFS yang mencari jarak terpendek, algoritma DFS dalam program ini dirancang untuk melakukan eksplorasi secara mendalam pada setiap percabangan rute yang mungkin ditemui hingga mencapai sasaran atau menemui jalan buntu. Asumsi utama yang mengikat perancangan algoritma ini adalah prinsip jalur sederhana, yakni sebuah rangkaian rute tidak diizinkan mengunjungi sel yang sama lebih dari satu kali dalam satu lintasan. Selain itu, program diasumsikan beroperasi pada ruang grid statis dengan pembatasan pergerakan ortogonal ke empat arah mata angin, serta jaminan bahwa titik awal dan tujuan selalu berada pada koordinat yang dapat diakses atau bernilai nol.

Dalam implementasinya, program ini mendayagunakan mekanisme rekursi fungsi sebagai pondasi utama penelusuran. Untuk efisiensi akses data dalam lingkup fungsi rekursif, program menggunakan variabel global seperti `R`, `C`, `sr`, `sc`, `fr`, `fc`, `grid`, `visited`, dan `totalPath`. Penggunaan variabel global ini bertujuan agar seluruh data dapat diakses langsung oleh fungsi DFS tanpa perlu dikirim sebagai parameter setiap kali fungsi dipanggil secara rekursif, yang menjaga performa alokasi memori stack. Fungsi `dfs` secara berulang memanggil dirinya sendiri guna menelusuri sel tetangga yang memenuhi kriteria validitas, yakni berada dalam batas dimensi grid, memiliki nilai jalan terbuka, dan belum pernah dikunjungi dalam rentang rute yang sedang diproses. Elemen kunci dari algoritma ini terletak pada teknik backtracking yang diimplementasikan melalui baris instruksi `visited[r][c] = false;` setelah proses eksplorasi ke semua arah pada sel tersebut rampung dilakukan. Instruksi ini berfungsi secara krusial untuk "membersihkan" jejak kunjungan, sehingga sel yang telah dilewati oleh satu rute dapat dipertimbangkan kembali sebagai bagian dari kombinasi rute alternatif lainnya yang memiliki cabang masuk berbeda.

Penelusuran backtracking pada program ini memiliki kompleksitas waktu dengan batas atas eksponensial, yakni $O(4^{R \times C})$. Lonjakan kompleksitas ini dipicu oleh sifat pencarian menyeluruh (exhaustive search) di mana program secara harfiah dipaksa untuk menjajal setiap permutasi jalur yang mungkin ada hingga mencapai titik tujuan,

mengingat status `visited` selalu diatur ulang setiap kali rekursi mengalami pemunduran. Sementara itu, kompleksitas ruang dari rancangan ini berbanding lurus dengan dimensi grid yakni $O(R \times C)$, yang dialokasikan untuk menampung matriks penanda `visited` serta kedalaman tumpukan memori rekursi (`call stack`) saat fungsi melakukan pemanggilan beruntun. Implementasi ini memberikan pembuktian metrik fungsional yang valid terhadap tuntutan persoalan, yakni kemampuan untuk menghitung total probabilitas jalur yang unik. Melalui teknik backtracking yang manipulatif terhadap status kunjungan sel, algoritma ini mampu memetakan setiap percabangan labirin secara presisi, menjamin bahwa tidak ada satu pun jalur sederhana yang luput dari kalkulasi dan laporan akhir program.

SOAL 5: Traversal

Time Limit: 1.0 s

Memory Limit: 64 MB

Deskripsi Soal

Diberikan N bilangan bulat. Masukkan seluruh bilangan tersebut ke dalam sebuah **Red-Black Tree** secara berurutan sesuai input. Jika terdapat bilangan duplikat, bilangan tersebut diabaikan dan tidak dimasukkan kembali ke tree..

Setelah RBTree terbentuk, diberikan Q query. Setiap query meminta salah satu hasil traversal berikut.

- **PREORDER**, yaitu urutan root, subtree kiri, lalu subtree kanan.
- **INORDER**, yaitu urutan subtree kiri, root, lalu subtree kanan.
- **POSTORDER**, yaitu urutan subtree kiri, subtree kanan, lalu root.
- **ALL**, yaitu mencetak ketiga traversal sekaligus.

Tugas Anda adalah mencetak hasil traversal sesuai query yang diberikan.

Format Input

Input diberikan dalam format berikut.

N
 $A_1 A_2 \dots A_N$
 Q
 T_1
 T_2
 $\dots$
 T_Q

Keterangan:

- N adalah banyaknya bilangan yang akan dimasukkan ke RBTree.
- A_i adalah bilangan ke- i .
- Q adalah banyaknya query traversal.
- T_i adalah jenis traversal yang diminta.

Format Output

Untuk setiap query, cetak hasil traversal sesuai format berikut.

[Preorder] : daftar_angka

[Inorder] : daftar_angka

[Postorder] : daftar_angka

Jika query adalah ALL, cetak ketiga baris traversal dengan urutan Preorder, Inorder, lalu Postorder.

Constraints

- $0 \leq N \leq 1000$
- $-1000000000 \leq A_i \leq 1000000000$
- $1 \leq Q \leq 20$
- T_i hanya salah satu dari PREORDER, INORDER, POSTORDER, atau ALL.

Contoh Kasus

Input	Output
7	[PREORDER] : 50 30 20 40 70 60 80
50 30 70 20 40 60 80	[INORDER] : 20 30 40 50 60 70 80
4	[POSTORDER] : 20 40 30 60 80 70 50
PREORDER	[PREORDER] : 50 30 20 40 70 60 80
INORDER	[INORDER] : 20 30 40 50 60 70 80
POSTORDER	[POSTORDER] : 20 40 30 60 80 70 50
ALL	

Penjelasan Contoh

Bilangan dimasukkan ke RBTree secara berurutan. Nilai 50 menjadi root. Nilai yang lebih kecil dari 50 masuk ke subtree kiri, sedangkan nilai yang lebih besar masuk ke subtree kanan.

Hasil INORDER pada RBTree selalu menghasilkan urutan nilai dari kecil ke besar. Pada contoh ini, hasil inorder adalah 20 30 40 50 60 70 80, sehingga struktur RBTree yang dibentuk sudah konsisten dengan aturan RBTree

A. Source Code

Tabel 5 Source Code file soal5.cpp

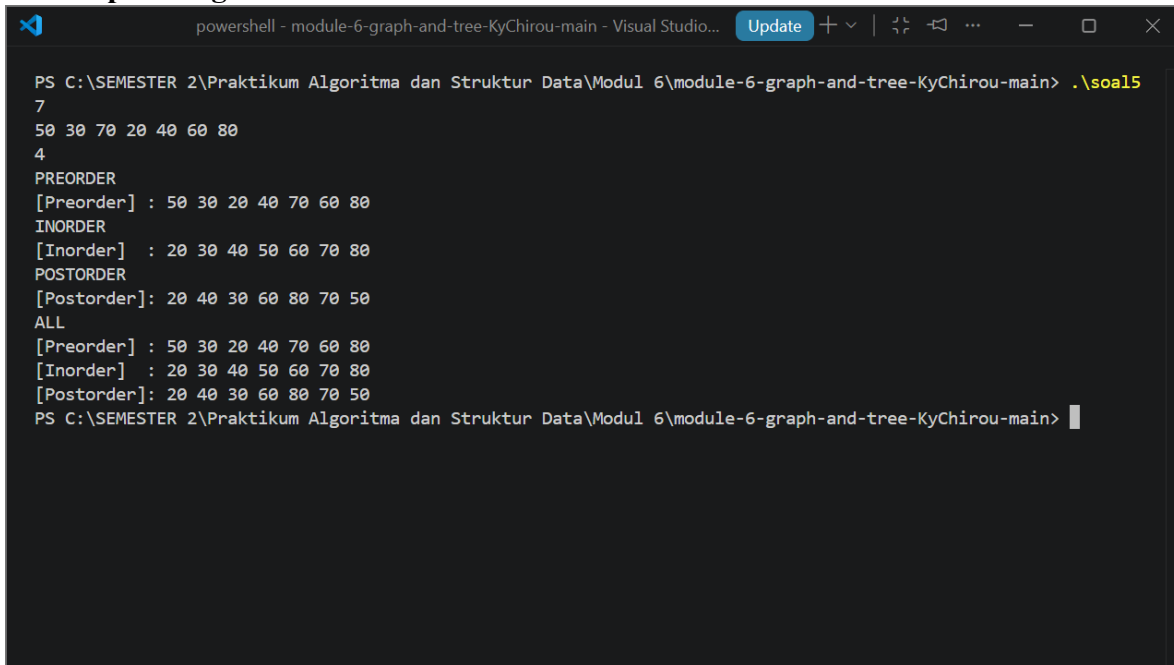
```
1  #include <iostream>
2  #include <string>
3  #include "RedBlackTree.h"
4  using namespace std;
5
6  void printPreorder(const RedBlackTree::Node* node) {
7      if (node == nullptr || node->isNil) return;
8      cout << node->key << " ";
9      printPreorder(node->left);
10     printPreorder(node->right);
11 }
12
13 void printInorder(const RedBlackTree::Node* node) {
14     if (node == nullptr || node->isNil) return;
15     printInorder(node->left);
16     cout << node->key << " ";
17     printInorder(node->right);
18 }
19
20 void printPostorder(const RedBlackTree::Node* node) {
21     if (node == nullptr || node->isNil) return;
22     printPostorder(node->left);
23     printPostorder(node->right);
24     cout << node->key << " ";
25 }
26
27 int main() {
28     int N;
29     if (!(cin >> N)) return 0;
30
31     RedBlackTree tree;
32
33     for (int i = 0; i < N; ++i) {
34         int val;
35         cin >> val;
36         if (!tree.contains(val)) {
37             tree.insert(val);
38         }
39     }
40
41     int Q;
42     cin >> Q;
43
44     for (int i = 0; i < Q; ++i) {
45         string query;
```

```

46         cin >> query;
47
48         if (tree.empty()) {
49             cout << "Tree kosong. Tidak ada yang bisa
ditampilkan.\n";
50             continue;
51         }
52
53         if (query == "PREORDER" || query == "ALL") {
54             cout << "[Preorder] : ";
55             printPreorder(tree.root());
56             cout << "\n";
57         }
58
59         if (query == "INORDER" || query == "ALL") {
60             cout << "[Inorder] : ";
61             printInorder(tree.root());
62             cout << "\n";
63         }
64
65         if (query == "POSTORDER" || query == "ALL") {
66             cout << "[Postorder]: ";
67             printPostorder(tree.root());
68             cout << "\n";
69         }
70     }
71
72     return 0;
73 }

```

B. Output Program



```
PS C:\SEMESTER 2\Praktikum Algoritma dan Struktur Data\Modul 6\module-6-graph-and-tree-KyChirou-main> .\soal5
7
50 30 70 20 40 60 80
4
PREORDER
[Preorder] : 50 30 20 40 70 60 80
INORDER
[Inorder] : 20 30 40 50 60 70 80
POSTORDER
[Postorder]: 20 40 30 60 80 70 50
ALL
[Preorder] : 50 30 20 40 70 60 80
[Inorder] : 20 30 40 50 60 70 80
[Postorder]: 20 40 30 60 80 70 50
PS C:\SEMESTER 2\Praktikum Algoritma dan Struktur Data\Modul 6\module-6-graph-and-tree-KyChirou-main>
```

Gambar 5 Screenshot Hasil Pengujian file soal5.cpp

C. Pembahasan

Kode ini dirancang untuk mengimplementasikan teknik penelusuran (traversal) pada struktur data Red-Black Tree (RBT). RBT merupakan variasi dari Binary Search Tree yang dilengkapi dengan aturan pewarnaan simpul (merah dan hitam) untuk menjamin keseimbangan tinggi pohon, sehingga operasi pencarian dan penyisipan dapat berlangsung secara efisien dalam waktu logaritmik. Program ini menerima input berupa sekumpulan bilangan bulat yang kemudian disusun ke dalam struktur RBT, diikuti dengan pemrosesan query penelusuran untuk menampilkan data berdasarkan urutan Preorder, Inorder, Postorder, atau ketiganya sekaligus (ALL). Asumsi utama dalam perancangan algoritma ini adalah keharusan menjaga integritas keunikan data, di mana setiap nilai duplikat akan diabaikan selama proses penyisipan agar struktur pohon tetap valid sesuai aturan RBT.

Dalam implementasinya, program menggunakan kelas `RedBlackTree` yang didukung oleh struktur `Node` sebagai penyimpan data kunci, atribut warna, serta pointer relasional ke anak (child) dan orang tua (parent). Penggunaan node sentinel (`isNil`) diterapkan sebagai pengganti nilai null guna menyederhanakan logika proses balancing pada saat penyeimbangan pohon. Proses penyisipan data diawali dengan pemeriksaan

eksistensi nilai melalui fungsi `contains()` untuk menjamin keunikan data. Jika nilai belum ada, prosedur `insert()` dijalankan layaknya penempatan pada Binary Search Tree biasa, di mana setiap node baru disisipkan sebagai leaf berwarna merah. Setelah penyisipan, algoritma memicu fungsi `fixInsert()` untuk melakukan restorasi keseimbangan melalui teknik recoloring atau rotasi (`rotateLeft` dan `rotateRight`), yang memastikan aturan pewarnaan RBT tetap terjaga dan root selalu berwarna hitam.

Setelah struktur pohon terbentuk secara utuh, program memproses query penelusuran dengan memanfaatkan metode rekursif pada fungsi `printPreorder`, `printInorder`, dan `printPostorder`. Metode Preorder mengeksekusi kunjungan dengan pola akar-kiri-kanan, Inorder dengan pola kiri-akar-kanan, dan Postorder dengan pola kiri-kanan-akar. Algoritma ini menjamin setiap simpul dikunjungi tepat satu kali, yang kemudian dicetak ke layar sesuai dengan query yang diminta. Secara analitis komputasional, proses pembentukan pohon memiliki kompleksitas waktu $O(N \log N)$ karena setiap operasi penyisipan ke dalam pohon yang seimbang memerlukan waktu $O(N \log N)$, yang dilakukan sebanyak N kali. Sementara itu, untuk setiap query penelusuran, kompleksitas waktu yang dibutuhkan adalah $O(N)$ karena sistem harus menelusuri seluruh simpul yang ada. Secara akumulatif, kompleksitas waktu program ini adalah $O(N \log N + Q \times N)$, dengan N merepresentasikan jumlah simpul dan Q sebagai total query. Dari sisi kompleksitas ruang, program membutuhkan memori sebesar $O(N)$ untuk menyimpan seluruh simpul pohon beserta struktur penunjang penelusurannya. Pembuktian metrik ini menunjukkan bahwa metode traversal yang diimplementasikan sangat efektif dalam mengubah struktur hierarki pohon menjadi urutan linier yang teratur, sekaligus memvalidasi bahwa seluruh data telah terorganisir dengan benar dalam struktur Red-Black Tree.

TAUTAN GIT

<https://github.com/DSA26-ULM/module-6-graph-and-tree-KyChirou>